알카노이드 온라인 게임화 추진 계획서

한국공학대학교



과목명	네트워크 게임 프로그래밍		
팀	4팀 (★ : 조장)		
	2022182034	2019182018	2022182021
	★임성진	박효근	송광민
애플리케이션 설명	1986년에 TAITO에 제작된 벽돌깨기 게임의 원조라고 할 수 있는 알카노이드를 윈도우 프로그래밍으로 재현한 프로그램으로 본 프로젝트의 원본은 알카노이드의 많은 버전들 중 NES판 알카노이드의 북미판. 그 중에서도 초반의 3개의 스테이지만 구현되어 있다.		
온라인화 기획	1. 우선 윈도우 프로그래밍 프로젝트 당시에 제작한 프레임워크가 팀 프로젝트에 적합하지 않고 개발에 비효율적이라 판단. 프레임워크가 잘 구축되어 있는 OpenGL 시스템으로 애플리케이션을 Migration, Refactoring 한다. 2. 원작 <알카노이드>에는 멀티플레이 모드가 없고 후속작인 <알카노이드 DOH>에는 2가지 멀티플레이 모드가 있었다. 하나는 플레이어가 번갈아 목숨을 다 사용할 때까지 누가 더 많은 스코어를 획득하는지, 또는 더 많은 스테이지를 클리어 하는지 겨루는 방식으로 '번갈아' 플레이한다는 내용 때문에 본 과제에는 적합하지 않다. 또 하나는 배틀 모드로 실시간으로 플레이어가 공을 주고 받으며 겨루는 Pong과도 비슷한 탁구형 게임이다. 이쪽은 실시간 동시 플레이로 진행되기 때문에 본 과제에 적합하지만 윈도우 프로그래밍으로 만든 프로그램과는 방향이 많이 달라져버린다. 때문에 이 2가지 모드를 적절히 섞는 방향으로 가기로 했다. 테트리스의 멀티플레이 모드 처럼 플레이어의 화면에 자신의 화면과 상대 플레이어 화면을 둘 다 띄우고 실시간으로 상대방의 화면을 보며 스테이지를 먼저 클리어하는 대결 방식의 멀티플레이를 구현한다.		

애플리케이션 상세 설명

1. 알카노이드의 기본 틀



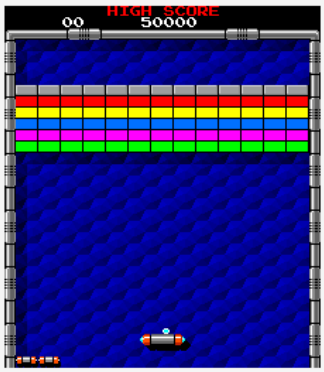
주인공 '바우스' | 레이저 바우스. 아이템 문단 참고.

바우스 : 플레이어 캐릭터. 공을 튕기는 판. 레이저 아이템을 먹고 레이저 바우스가 되면 레이저를 쏴서 블록이나 AI를 파괴할 수 있다.

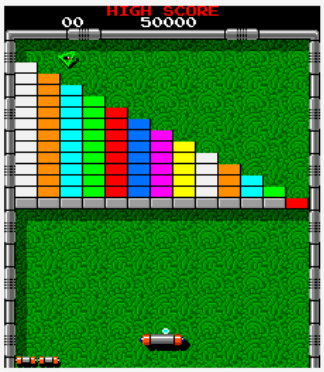
아이템	
이름	효과
Player	목숨 추가
Laser	레이저 바우스로 변형
Enlarge	바우스의 길이가 1.5배가 됨
Catch	공을 잠깐 잡을 수 있음
Slow	공이 느려짐
Disruption	공이 3개로 분열함

블록	
색깔	점수
White	50
Orange	60
Cyan	70
Green	80
Red	90
Blue	100
Magenta	110
Yellow	120
Sliver (Hard Wall)	(스테이지 레벨) X 50
Gold (Immortality Wall)	파괴 불능

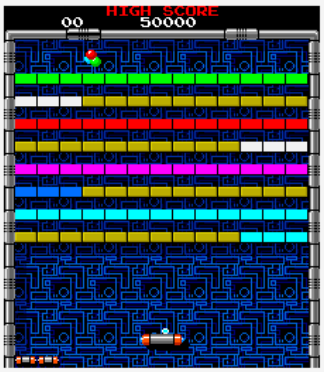
애플리케이션 상세 설명



스테이지1



스테이지2



스테이지3

3개의 스테이지로 구성하며 가장 기본형인 스테이지1, 계단형의 스테이지2, 행과 행 사이에 공간이 있는 스테이지3로 이루어져 있다.

2. 멀티플레이 기획



(예시 : 뿌요뿌요 테트리스)

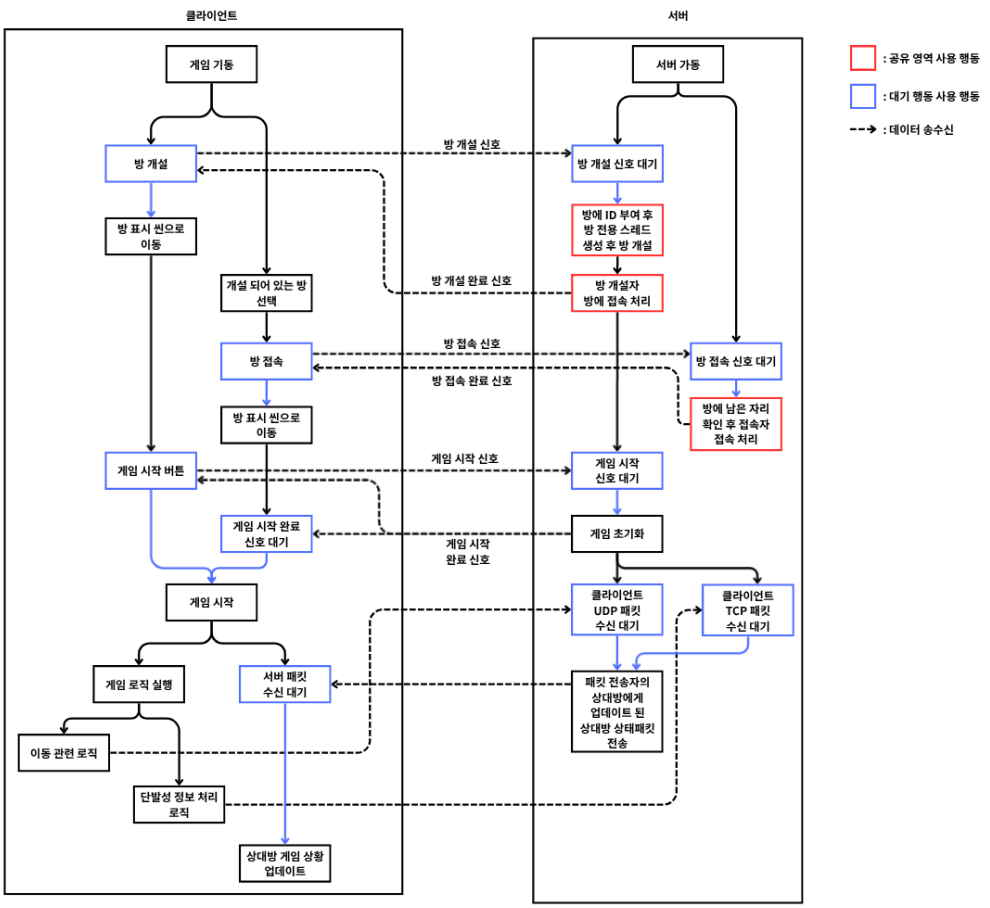
위의 사진처럼 각 플레이어의 화면을 양 옆에 두고 가운데는 서로의 스코어를 표시한다. 스테이지3를 두 사람이 모두 클리어할 때까지 지속되며 원본 게임과 다르게 공을 튕기지 못해도 목숨을 잃지 않는 무한 목숨 시스템이다. 단, 목숨을 한 번 잃을 때마다 스코어에서 10점이 차감된다.

멀티 플레이는 호스트 클라이언트가 방을 만들면 서버 목록에 표시되고 게스트 클라이언트가 방에 접속하고 클라이언트 둘이 준비 신호를 보내면 게임을 시작한다.

클라이언트에서 서버에 정보를 보내면 해당 정보를 상대방 클라이언트에도 전달하는 방식으로 구현한다.

게임 시작시, 아이템은 원작과 다르게 랜덤 획득이 아닌 게임 시작단계에서 위치가 정해진다.

High-Level 디자인



서버에 양측 플레이어의 게임 현황 정보를 저장하고 클라이언트가 서버에서 일정주기로 상대방의 플레이 현황을 요청하고 읽어오는 방식으로 상대방의 플레이가 보이도록 구현.

Low-Level 디자인

[방 개설, 접속과 관련된 기능과 스레드 설계]

[클라이언트]

- 방 개설 함수
- 방 접속 함수
- 게임 시작 신호 전송 함수(Game Start 기능)
- 게임 시작 대기 신호 함수(Ready 기능)

[서버]

- 방 개설 스레드
  - 개설 처리 함수
- 방 접속 스레드
  - 접속 처리 함수
- 개별 방 스레드
  - 게임 시작 대기 함수

방 개설과 관련된 정보 처리는 모두 TCP 프로토콜로 처리하고 방이 개설 되고 게임을 시작하고 나서부터는 TCP, UDP 프로토콜로 송수신할 정보를 나누는 것으로 서로의 영역에 간섭하지 않도록 설계했다.

[TCP, UDP 프로토콜에 따른 송수신할 데이터 설계]

프로토콜	송수신 할 정보
TCP	블록 피격 · 파괴, 아이템 획득 · 해제, 소유 공 개수, 리스폰 여부, 현재 스테이지, 점수
UDP	이동과 관련된 정보를 포함한 매 프레임 변화하는 데이터

확실하게 전달되어야 하는 상대적으로 데이터 송수신 빈도가 낮은 데이터의 경우에는 TCP 프로토콜로 데이터를 송수신하고 매 프레임 갱신되며 한 프레임 정도는 놓쳐도 게임에 큰 문제가 없는 데이터는 UDP 프로토콜로 데이터를 송수신 하도록 설계했다.

[TCP 데이터 송수신 이벤트 및 패킷 설계]

모든 이벤트는 같은 구조체로 데이터를 송수신 한다. Event 값과 인자 3개의 의미만 다르고, 모든 이벤트에 보낸 쪽의 Score를 담아서 서버가 함상 최신 Score 상태이도록 한다.

공통인자 : timeMs(클라이언트 게임시간), event(무슨 일인지), score(총점).

항상 16바이트이므로 recv(sock, &pkt, 16, MSG_WAITALL)로 받는다

- 클라이언트에서 서버로 보내는 패킷

EventType	**보내는 때**	**arg8**	**arg16**	**arg32**
BLOCK_HIT	Silver 블록이 맞았지만 안 깨짐	-	블록 인덱스	남은 내구도
BLOCK_BROKEN	블록 파괴	-	블록 인덱스	얻은 점수
ITEM_GET	캡슐 획득	아이템 종류	-	-
ITEM_END	아이템 효과 해제	아이템 종류	-	-
BALL_LOST	공을 모두 놓쳐 리스폰 대기 시작	-	-	대기 시간 (ms)
RESPAWN	새 공 지급 (Player 아이템이면 대기 없이 바로)	-	-	-

STAGE_CLEAR	스테이지 클리어 (3이면 모든 스테이지 완료)	스테이지 번호	남은 공 수	보너스 점수
STAGE_START	다음 스테이지 시작	스테이지 번호	-	-

- 서버에서 클라이언트로 보내는 패킷

패킷	보내는 때	내용
SC_TIME_SYNC	5초마다 (제안)	남은 시간(초). 클라이언트 타이머 표시를 서버 시간에 맞춘다
SC_GAME_END	제한시간 종료, 또는 둘 다 스테이지 3 클리어	승자(0 / 1 / 무승부), 사유, 두 사람의 점수
SC_OPP_LEFT	게임 중 상대 연결이 끊김	없음. 바로 뒤에 SC_GAME_END(남은 사람 승리)를 보낸다

[UDP 데이터 송수신 로직 및 함수 설계]

1. 클라이언트 A에서 입력을 감지하고 바우스의 좌표를 갱신한다.
2. 현재 자신의 로컬의 바우스의 좌표를 서버에 전송한다. (UDP_ONLYPOS)
3. 서버는 이동된 바우스의 좌표를 갱신하고 게임월드의 update를 한다.
4. 업데이트 중 좌표가 변경된 오브젝트의 번호와, 상태가 변경된 오브젝트의 번호를 각각의 vector에 저장한다
5. 좌표가 변경된 오브젝트의 메시지와 상태가 변경된 오브젝트의 메시지를 각각의 스레드로 전송한다. (UDP_ONLYPOS), (UDP_CLASSDATA)
6. 1/5초(임시)마다 서버에서 모든 오브젝트의 상태를 전부 클라이언트로 전송해서 동기화한다. (UDP_CLASSDATA)

* 5.에서 데이터를 전송할 때 A클라이언트에 전송하는 스레드의 경우, A의 바우스의 메시지는 무시하고 전송하고, 6.에서 보낼 때 동기화한다

[클라이언트가 사용하는 함수]

1. 바우스 이동 입력 감지
2. 입력에 따라 로컬 바우스 좌표 갱신
3. 자신의 바우스 좌표를 UDP_ONLYPOS로 서버에 전송

4. 서버에서 전송한 UDP 패킷 수신 스레드
5. 수신한 UDP_ONLYPOS 좌표를 오브젝트에 반영
6. 수신한 UDP_CLASSDATA 정보를 오브젝트에 반영

[서버가 사용하는 함수]
1. 클라이언트 A, B의 좌표 패킷 수신
2. 수신한 좌표를 서버의 바우스에 적용
3. 서버 게임 월드 업데이트
4. 좌표가 변경된 오브젝트 ID를 positionChangedVector에 저장
5. 상태가 변경된 오브젝트 ID를 stateChangedVector에 저장
6. positionChangedVector의 오브젝트를 UDP_ONLYPOS로 전송
7. stateChangedVector의 오브젝트를 UDP_CLASSDATA로 전송
8. 전송 처리된 변경 목록 초기화
9. 0.2초마다 전체 동기화 실행
10. 게임 월드의 모든 오브젝트 상태 수집해서 stateChangedVector에 저장

팀원 별 역할분담

임성진	알카노이드 리팩토링, 호스트, 게스트 시스템
박효근	TCP 통신 부분
송광민	UDP 통신 부분

개발환경

기본 구현 라이브러리 : OpenGL
네트워크 기능 API : Winsock
IDE : Visual Studio 2026
개발 운영체제 : Windows 11
사용 언어 : C++ v145

개발일정

임성진:

날짜	할 일
11/1	
11/2	
11/3	
11/4	리팩토링(스테이지1, 2, 3 구현)
11/5	리팩토링(바우스, 공 튀기기 구현)
11/6	리팩토링(UI구현, 벽돌 파괴시 스코어 상승 구현)

11/7	리팩토링(Enlarge, Slow, Catch, Disruption 아이템 구현)
11/8	리팩토링(Laser 아이템 구현), 리팩토링 버그 수정
11/9	
11/10	
11/11	
11/12	
11/13	
11/14	방 선택 씬, UI 구현
11/15	[클라이언트] 방 개설 함수, 방 접속 함수 구현
11/16	[서버] 개설 처리 함수, 접속 처리 함수 구현
11/17	
11/18	
11/19	
11/20	
11/21	[클라, 서버] 게임 시작 대기, 게임 시작 구현
11/22	메인 화면 구현, 사운드 재생
11/23	방 접속 시스템 버그 확인 및 수정
11/24	
11/25	
11/26	
11/27	
11/28	
11/29	
11/30	
12/1	
12/2	
12/3	
12/4	네트워크 기믹들 최종 점검
12/5	
12/6	
12/7	
12/8	

박효근:

날짜	할 일
11/1	
11/2	게임 중 상대에게 알려야 할 정보 목록 정리
11/3	패킷 구조, 게임 소켓 포트, 게임 소켓 연결 절차 확정
11/4	
11/5	패킷 구조체와 송수신 함수 완성
11/6	로비 파트와 게임 시작 시 방 번호, 플레이어 번호 전달 방식 확정
11/7	
11/8	
11/9	테스트용 서버, 클라이언트 완성, 패킷 주고받기 확인
11/10	
11/11	
11/12	서버: 게임 소켓 접속과 방 매칭 완성
11/13	UDP 파트와 플레이어 구분 방식 합의
11/14	
11/15	서버: 이벤트를 받아 플레이어 상태(점수 · 스테이지) 갱신 완성
11/16	
11/17	서버: 상대에게 중계 완성, 테스트 클라이언트 2개로 확인
11/18	
11/19	클라이언트: 게임 소켓 연결과 이벤트 송신 완성
11/20	게임 로직 파트와 이벤트 보내는 위치 연결 확인
11/21	
11/22	클라이언트: 수신 스레드와 상대 화면 반영 완성
11/23	
11/24	제한시간 타이머와 남은 시간 알림 완성
11/25	
11/26	승패 판정과 결과 화면 전달 완성
11/27	로비 -> 게임 -> 결과 통합
11/28	
11/29	연결 끊김 · 상대 미접속 처리 완성
11/30	
12/1	UDP 파트와 합쳐 두 PC 대전 테스트

12/2	
12/3	예외 상황 테스트(강제 종료, 이상한 데이터)와 버그 수정 완료
12/4	서로 다른 네트워크에서 대전 테스트
12/5	
12/6	버그 수정 · 코드 정리 완료
12/7	보고서 담당 파트 작성
12/8	최종 점검 · 제출

송광민:

날짜	할 일
11/1	
11/2	키보드 입력 처리
11/3	로컬 좌표에 입력 반영
11/4	좌표 udp로 전송, 서버에서 출력
11/5	
11/6	
11/7	
11/8	서버 내의 바우스 좌표에 반영
11/9	
11/10	
11/11	positionChangedVector에 정보 삽입
11/12	positionChangedVector 안전하게 clear()
11/13	
11/14	
11/15	
11/16	
11/17	positionChangedVector 정보 송신
11/18	positionChangedVector 정보 수신
11/19	클라이언트 로컬 오브젝트 좌표 수정
11/20	
11/21	
11/22	
11/23	
11/24	stateChangedVector에 정보 삽입
11/25	stateChangedVector 안전하게 clear()

	11/26	
	11/27	
	11/28	stateChangedVector정보 송신
	11/29	stateChangedVector정보 수신
	11/30	클라이언트 로컬 오브젝트 좌표 수정
	12/1	
	12/2	
	12/3	
	12/4	전체 동기화(stateChangedVector에 삽입)
	12/5	전체 동기화(stateChangedVector 송/수신)
	12/6	
	12/7	
	12/8	